Name: Aadesh Khande
PRN: 202401100090
Div: B Batch B1

Practical 2: Design and develop any suitable web application to demonstrate the capabilities of Javascript. Use JSON and Array for storing the data, before saving the data, validate it and show proper error messages.

Deployed / Application Link: javascript-assignment-sable-one.vercel.app
Github Repository for code: Codes

Aim: To design and develop an interactive, clean, and responsive web application (Student Expense Tracker) using HTML5, CSS3, and Vanilla JavaScript (ES6+) incorporating a **shadcn Stepper Component** and **Google Sans** typography, to demonstrate client-side JavaScript capabilities, in-memory structured data management using JavaScript Arrays and JSON objects, step-wise input validation before saving with instant error messages, and dynamic DOM event handling.

Objectives:

- Each student must develop an individual interactive web application.
- Use HTML, CSS and JavaScript.
- Use Arrays and JSON for storing application data.
- Validate the user input before saving the data.
- Display proper error/validation messages for invalid input.
- The application must be interactive and respond to user actions/events.

Theory

1. JavaScript Dynamic Capabilities & Event-Driven DOM Manipulation:

JavaScript is an asynchronous, event-driven scripting language executed within the browser's JavaScript engine (e.g., V8). Through the Document Object Model (DOM) API, JavaScript interacts dynamically with HTML elements in response to user actions (clicks, keyboard input, change events). Modern ES6+ language features such as arrow functions, template literals, destructuring, and `addEventListener` enable responsive multi-step wizard user interfaces without page reloads.

2. In-Memory Data Structures: JavaScript Arrays & Functional Pipelines:

Arrays in JavaScript are dynamic, ordered collections that provide built-in methods for data processing:

- **push() / unshift():** Appends newly validated expense objects to the in-memory array.
- **reduce():** Aggregates array elements in a single functional pass to compute the cumulative sum of expenses: `expenses.reduce((sum, e) => sum + e.amount, 0)`.
- **filter():** Produces filtered subsets of the array matching search keywords and category dropdown selections without mutating the source array, as well as filtering out deleted records.
- **some():** Performs an efficient O(n) check across the array to determine if an identical expense (same title, amount, and date) already exists, preventing duplicate records.
- **splice() & findIndex():** Locates and removes specific entries by unique ID upon deletion.

3. JSON (JavaScript Object Notation): Serialization & Storage:

- **JSON.stringify():** Serializes in-memory JavaScript Arrays and candidate objects into formatted JSON strings, enabling live data inspection and file export.
- **JSON.parse():** Deserializes JSON strings back into live JavaScript objects, enabling seamless data restoration from the browser's `localStorage`.

4. shadcn Stepper Component & Client-Side Validation Architecture:

- **Step-Wise Progression:** Enforces validation barriers at each step before allowing progression: Step 1 validates Title and Category; Step 2 validates Amount and Date; Step 3 presents review confirmation and duplicate check.
- **Title Validation:** Required, 3 to 30 characters, must not be numbers only (verified via `/^\d+$/` regex test).
- **Amount Validation:** Required, numeric, strictly > 0 and ≤ 1,00,000.
- **Date Validation:** Required, format verified, and strictly prevented from being set to any date in the future.
- **Duplicate Prevention:** Scans the array using `expenses.some()` to display a warning badge prior to final commit.

JavaScript Features & UI Components Used:

Component / Feature	Purpose & Implementation in Student Expense Tracker
shadcn Stepper Component `Multi-Step UI`	Progressive 3-step wizard with active step numbers, completed checkmarks, and progress lines.
Google Sans Typography `Google Sans`	Clean, modern font family applied throughout layout, form controls, and ledger table.
Master Array Store `expenses = [...]`	In-memory array storing structured JSON expense objects; primary source of truth.
Live Total Aggregator `Array.reduce()`	Dynamically calculates the cumulative expense sum in real-time.
Category Filter & Search `Array.filter()`	Filters ledger entries on-the-fly by category and search keyword.
Row Deletion `Array.filter()`	Removes individual expenses by ID when the delete button is clicked.
JSON Serialization `JSON.stringify()`	Formats in-memory records into indented JSON for live inspection and file export.
Step-Wise Validation Engine `Pure JavaScript`	Validates title length, numbers-only check, numeric bounds, category, and date at each step.
Duplicate Record Check `Array.some()`	Verifies uniqueness before appending entries to prevent duplicate costs.
Inline Visual Error Messages `DOM manipulation`	Contextual red error text dynamically displayed beneath fields via DOM manipulation.

Procedure / Implementation Steps:

1. **Project Setup & Layout Structure:**
   - Create `index.html` with semantic header, telemetry KPI cards, shadcn Stepper form container, test scenario pills, ledger table, and live JSON data console.
   - Configure modern fonts (*Google Sans* and *JetBrains Mono*) for clean typography and tabular numbers.
2. **Styling with Impeccable CSS & shadcn Stepper Architecture:**
   - Define CSS variables for dark zinc surfaces (`#09090b` canvas, `#121215` card), borders (`#27272a`), text contrast (≥ 4.5:1), and red error highlights (`#f87171`) in `styles.css`.
   - Implement shadcn stepper navigation with `.step-item`, `.step-indicator` (active circle with white background, completed circle with green checkmark ✓), and `.stepper-separator`.
3. **In-Memory Array & LocalStorage State Management:**
   - Declare `expenses = []` in `script.js` initialized with 5 benchmark student expenses (Lunch, Bus Pass, Reference Book, Stationery, WiFi).
   - Implement `loadExpenses()` and `saveExpenses()` to persist records to `localStorage` under key `student_expenses_v1`.
4. **Implementing the Stepper Logic & Step-Wise Validation Engine:**
   - Code `goToStep(step)` to toggle panels, update stepper badges, and populate the review card on Step 3.
   - Implement `validateStep1()` for Title (3–30 chars, no numbers-only) and Category.
   - Implement `validateStep2()` for Amount (₹1 to ₹1,00,000) and non-future Date.
   - Implement duplicate detection scanning `expenses.some()` for matching title, amount, and date displayed dynamically on Step 3.
5. **Building the Array Processing Pipeline & Table Rendering:**
   - Implement `renderTable()` using `filter()` to support search keywords and category dropdown filtering.
   - Implement `updateKPIs()` using `reduce()` to update total amount spent, total count, and most frequent category.
   - Generate dynamic `<tr>` elements with formatted amounts (₹), category badges, and delete action triggers.

6. **Interactive Actions, Candidate JSON Preview, and Export:**
   - Display candidate JSON payload on Step 3 prior to final submission.
   - Render live master JSON output via `JSON.stringify(expenses, null, 2)` inside a dedicated `<pre>` box.
   - Provide copy-to-clipboard and `.json` file download capabilities.

Validation Test Matrix & Verification Results:

Test #	Field / Constraint	Test Input Scenario	Validation Result	Application Behavior
TC-01	All Fields (Valid)	Title: `"College Textbook"`, Amount: `450`, Cat: `Study`, Date: Valid	**PASS (Accepted)**	Stepper advances to Step 3; review confirmed; appended to array.
TC-02	Title Too Short	Title: `"AB"` (< 3 characters) on Step 1	**PASS (Rejected)**	Step 1 halted; red border; error: *"Title must be at least 3 characters."*
TC-03	Title Numbers Only	Title: `"123"` (numbers only) on Step 1	**PASS (Rejected)**	Step 1 halted; red border; error: *"Title cannot consist of numbers only."*
TC-04	Amount Zero / Negative	Amount: `-50` on Step 2	**PASS (Rejected)**	Step 2 halted; red border; error: *"Amount must be greater than 0."*
TC-05	Amount Exceeds Max	Amount: `150000` (> ₹1,00,000) on Step 2	**PASS (Rejected)**	Step 2 halted; red border; error: *"Amount cannot exceed ₹1,00,000."*
TC-06	Category Unselected	Category: `""` on Step 1	**PASS (Rejected)**	Step 1 halted; red border; error: *"Please select a category."*
TC-07	Future Date	Date: `2029-10-15` on Step 2	**PASS (Rejected)**	Step 2 halted; red border; error: *"Date cannot be in the future."*
TC-08	Date Missing	Date: `""` on Step 2	**PASS (Rejected)**	Step 2 halted; red border; error: *"Expense date is required."*
TC-09	Duplicate Check	Same title, amount, and date on Step 3	**PASS (Rejected)**	Step 3 displays warning badge: *"Duplicate: An identical expense already exists!"*

Output Screenshot:

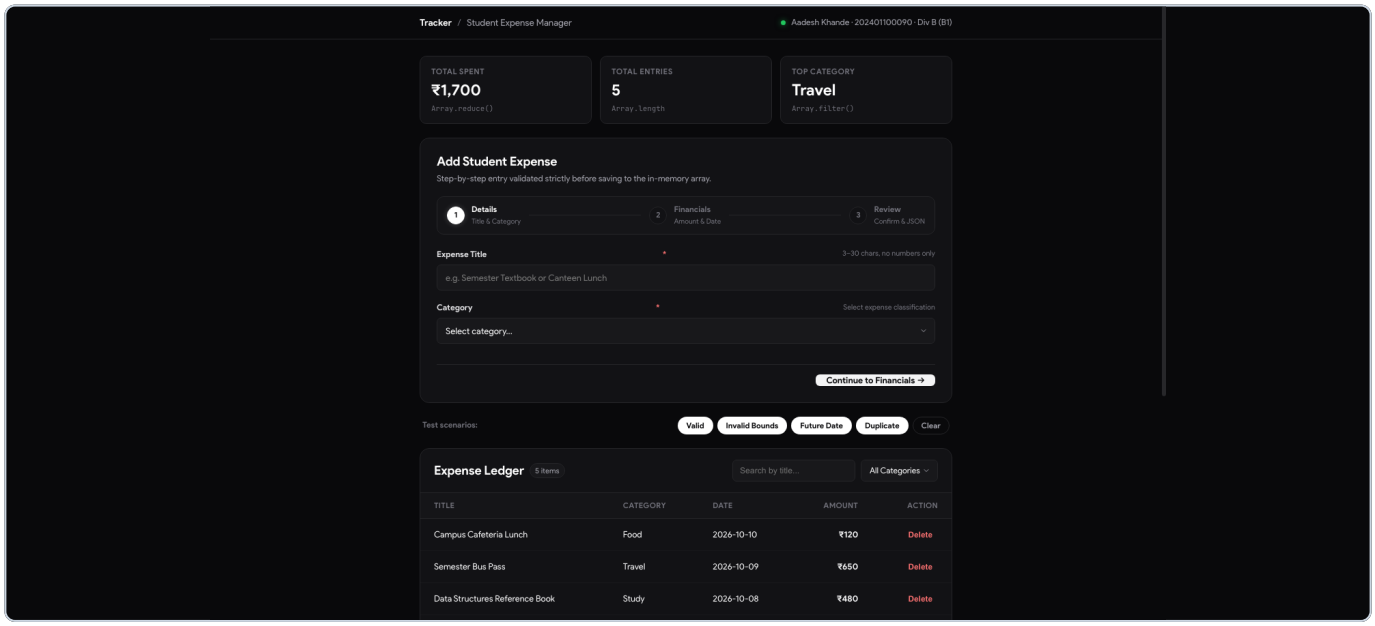


Figure 1: Student Expense Tracker Core Overview with shadcn Stepper Component (Step 1: Details & Google Sans Typography)

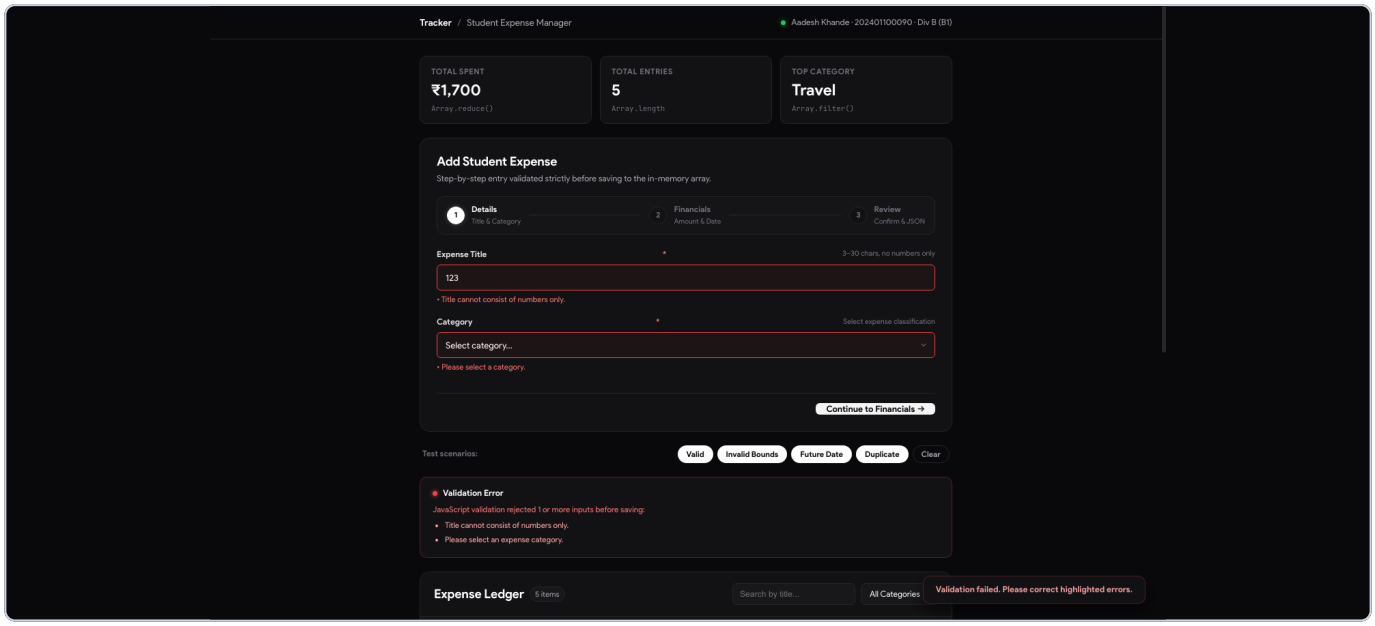


Figure 2: Real-Time Stepper Input Validation Failure State (Red Field Borders, Inline Warning Messages, and Diagnostics Card)

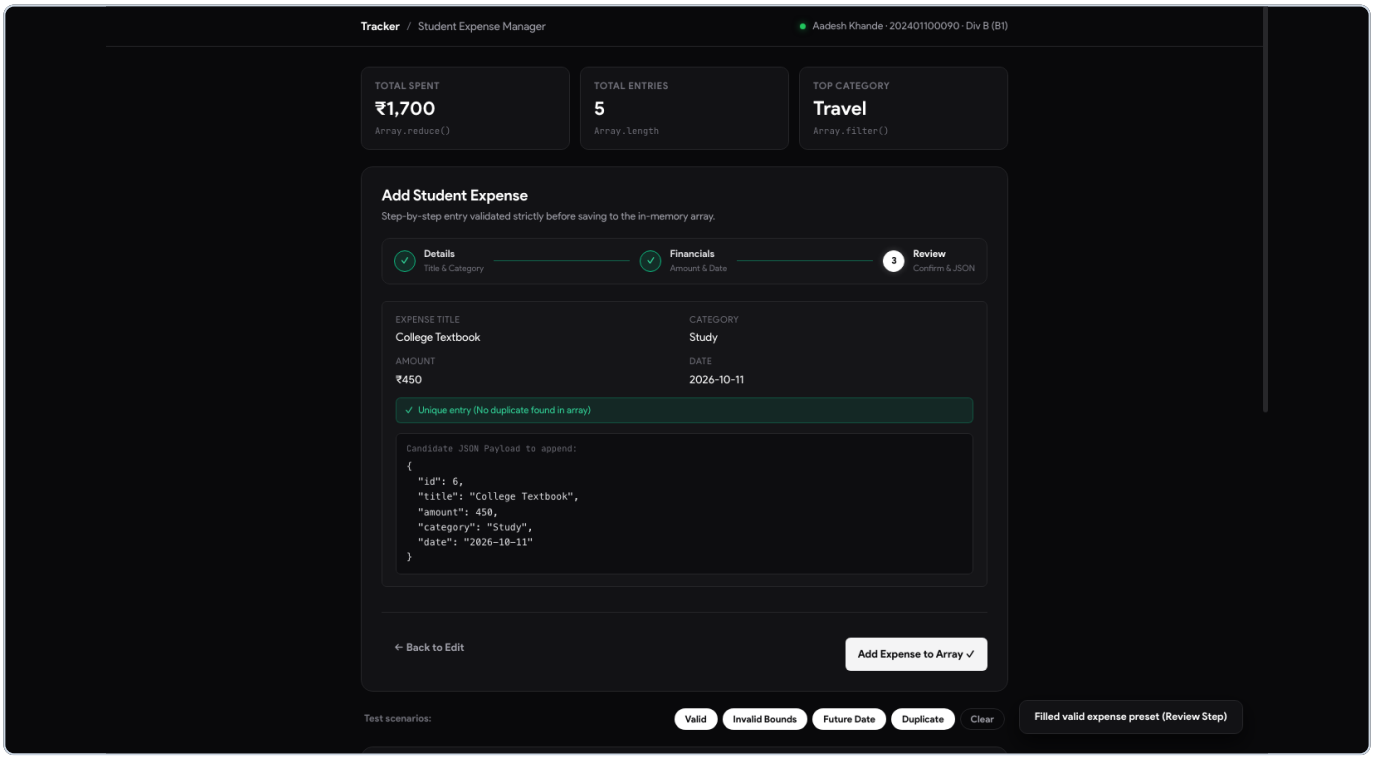


Figure 3: Stepper Step 3 Review Confirmation, Duplicate Prevention Check & Candidate JSON Payload Inspection

Conclusion & Practical Verification:

Practical 2 successfully demonstrates the dynamic capabilities of JavaScript, in-memory Array data processing pipelines, and JSON serialization. Incorporating a **shadcn-inspired Stepper Component** and **Google Sans** typography, the **Student Expense Tracker** provides an intuitive multi-step workflow. Before committing any new expense, the client-side validation engine strictly validates inputs at each step against string length, numbers-only pattern, numerical range bounds, category selections, non-future dates, and duplicate records, displaying clear red inline feedback upon invalid input. All practical requirements are fulfilled with an elegant, modern, and accessible user interface.

Validation Test Suite Status	**✓ All Test Scenarios Passed (100% Assertion Success Rate)**
In-Memory State Engine	JavaScript Array with JSON Object Persistence & LocalStorage Synchronization
Interactive Capabilities	shadcn Stepper Component, Google Sans Typography, Array Pipelines (filter, reduce, push, some), Live Search, JSON Inspector